How to write PHP Code?

- There are two ways of PHP programming pattern.
  1. Procedural/Core PHP
  2. Object Oriented

What is OOP?

- Object Oriented Programming is Coding Methodology / style / pattern.
  - Code more Modular and Reusable.
  - Well organized code
  - Easier to debug.
  - Best for medium to large website project.

Class and Object

- Classes are the blueprints of objects. One of the big differences between functions and classes is that a class contains both data (variables) and functions that form a package called an: 'object'.
- Class is a programmer-defined data type, which includes local methods and local variables.
- Class is a collection of objects. Object has properties and behavior.

Syntax: We define our own class by starting with the keyword **'class'** followed by the name you want to give your new class.

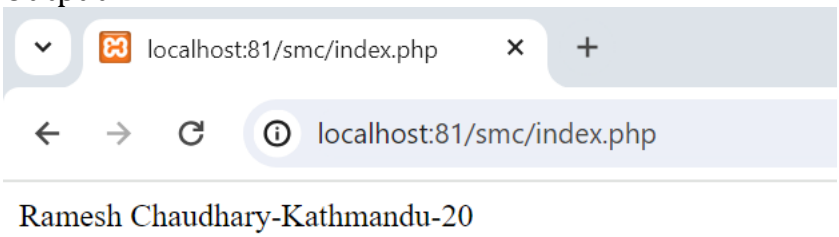
$this Keyword

- The **$this** keyword refers to the current object, and is only available inside methods.

Example: Example program to demonstrate class and object.

```
<?php
class person{
   public $name,$address,$age;
   function show(){
      echo $this->name."-".$this->address."-".$this->age;
   }
}
$p1=new person();
$p1->name="Ramesh Chaudhary";
$p1->address="Kathmandu";
$p1->age=20;
$p1->show();
?>
```

Output:

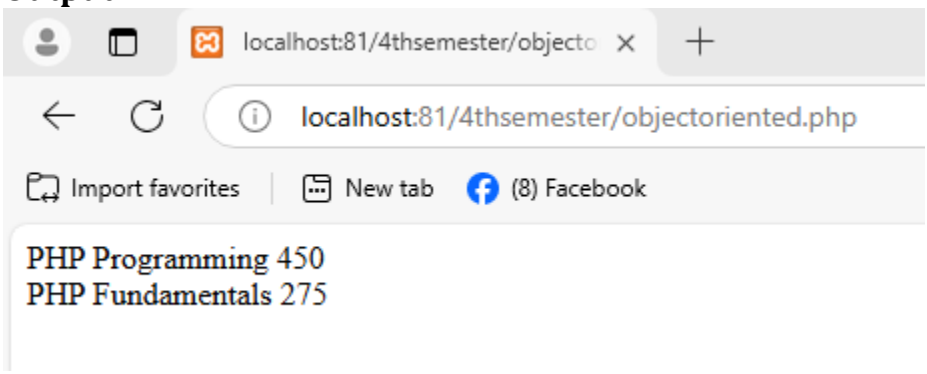


Example:

```
<?php
  class Book {

    /* Member variables */
    var $price;
    var $title;

    /* Member functions */
    function setPrice($par){
      $this->price = $par;
    }

    function getPrice(){
      echo $this->price ."<br>";
    }

    function setTitle($par){
      $this->title = $par;
    }

    function getTitle(){
      echo $this->title ."\n";
    }
  }

  $b1 = new Book;
  $b2 =new Book;

  $b1->setTitle("PHP Programming");
  $b1->setPrice(450);
  $b2->setTitle("PHP Fundamentals");
```

```
  $b2->setPrice(275);
  $b1->getTitle();
  $b1->getPrice();
  $b2->getTitle();
  $b2->getPrice();
?>
```

Output:

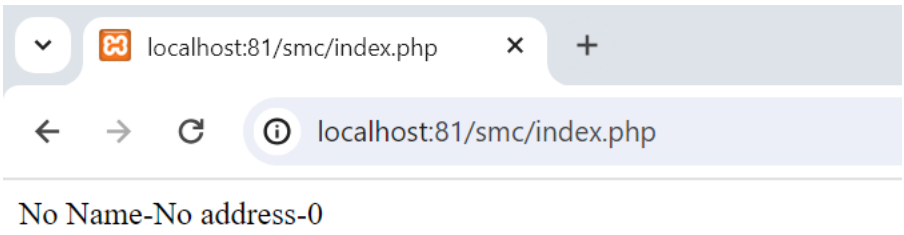


How to set default value

- In PHP, default values are used in various contexts, such as function arguments, variables, or object properties.

```
<?php
class person{
   public $name="No Name",$address="No address",$age=0;
   function show(){
      echo $this->name."-".$this->address."-".$this->age;
   }
}
$p1=new person();
/*
$p1->name="Ramesh Chaudhary";
$p1->address="Kathmandu";
$p1->age=20;
*/
$p1->show();
?>
```

Output:



Constructor in PHP

- A constructor is a special method that is automatically called when an object of a class is instantiated.
- It is typically used to initialize properties or execute any setup code required when the object is created.
- The constructor method is named __construct(). It is a reserved method that starts with two underscores.

Benefits of Using Constructors

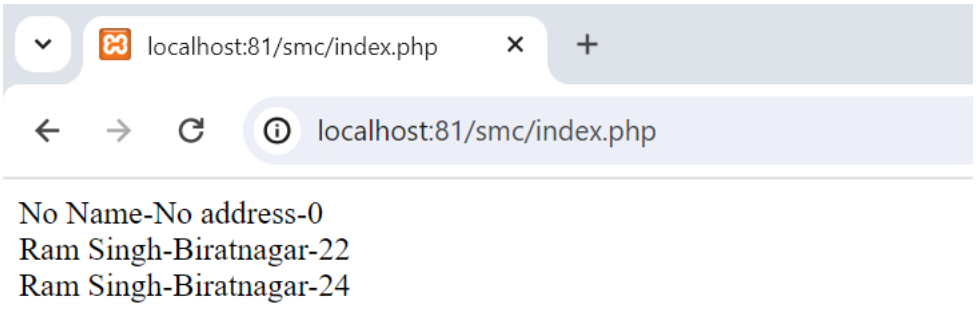
- **Automatic Initialization:** Ensures that properties are set up properly when an object is created.
- **Improved Code Organization:** Groups setup logic in a single, easy-to-find location.
- **Flexibility:** Supports passing parameters to customize the object at creation.

Example: Example program to demonstrate constructor in PHP.

```
<?php
class person{
   public $name,$address,$age;

  function    __construct($name="No Name",$address="No address",$age=0)
  {
   $this->name=$name;
   $this->address=$address;
   $this->age=$age;

  }
   function show(){
      echo $this->name."-".$this->address."-".$this->age."<br>";
   }
}
$p1=new person();
```

```
$p2=new person("Ram Singh","Biratnagar",22);
$p3=new person("Ram Singh","Biratnagar",24);
/*
$p1->name="Ramesh Chaudhary";
$p1->address="Kathmandu";
$p1->age=20;
*/
$p1->show();
$p2->show();
$p3->show();



?>
```

Output:

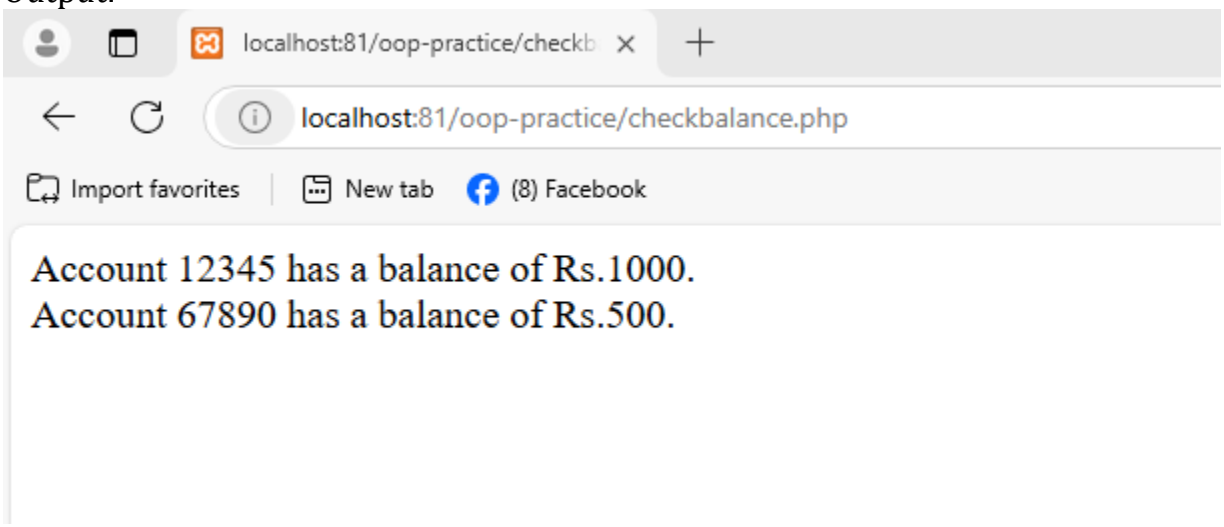


Example:

```
<?php
class BankAccount {
   public $accountNumber;
   public $balance;

   public function __construct($accountNumber, $balance) {
      if ($balance < 0) {
         die("Error: Initial balance cannot be negative.\n");
      }
      $this->accountNumber = $accountNumber;
      $this->balance = $balance;
   }

   public function displayBalance() {
```

```
        echo "Account $this->accountNumber has a balance of Rs.$this->balance.\n";
    }
}

$account1 = new BankAccount("12345", 1000); // Valid balance
$account1->displayBalance(); // Outputs: Account 12345 has a balance of Rs.1000.
echo("<br>");
$account2 = new BankAccount("67890", 500); // Error: Initial balance cannot be
negative.
$account2->displayBalance();
?>
```

Output:



Destructor in PHP

- In PHP, a **destructor** is a special method that is automatically invoked when an object is destroyed or goes out of scope. The primary purpose of a destructor is to release resources or perform cleanup tasks such as closing database connections or freeing memory.

  Syntax:

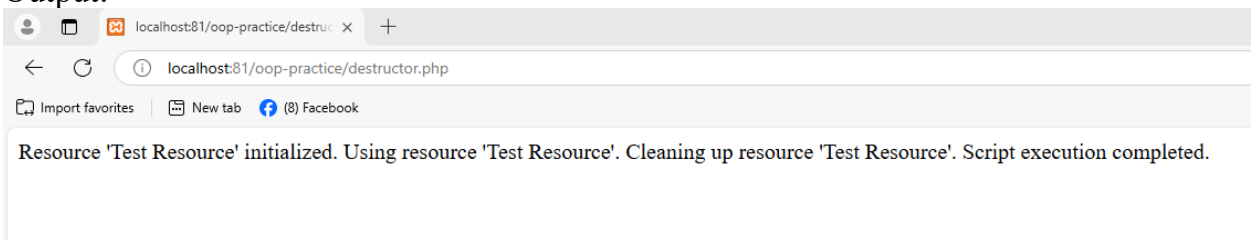
```
class ClassName {
    public function __destruct() {
        // Cleanup code
    }
}
```

Example:

```
<?php
```

```
class Demo {
    private $resource;

    public function __construct($resourceName) {
        $this->resource = $resourceName;
        echo "Resource '{$this->resource}' initialized.\n";
    }

    public function __destruct() {
        echo "Cleaning up resource '{$this->resource}'.\n";
    }

    public function useResource() {
        echo "Using resource '{$this->resource}'.\n";
    }
}

// Create an object
$demo = new Demo("Test Resource");

// Use the object
$demo->useResource();

// End of script (or unset object) triggers destructor
unset($demo);

echo "Script execution completed.\n";
?>
```

Output:



Inheritance

- ☞ Inheritance allows us to write the code only once in the parent, and then use the code in both the parent and the child classes.
- ☞ In order to declare that one class inherits the code from another class, we use the **extends** keyword.

```
class Parent {
// The parent's class code
}
class Child extends Parent {
// The child can use the parent's class code
}
```

Note:

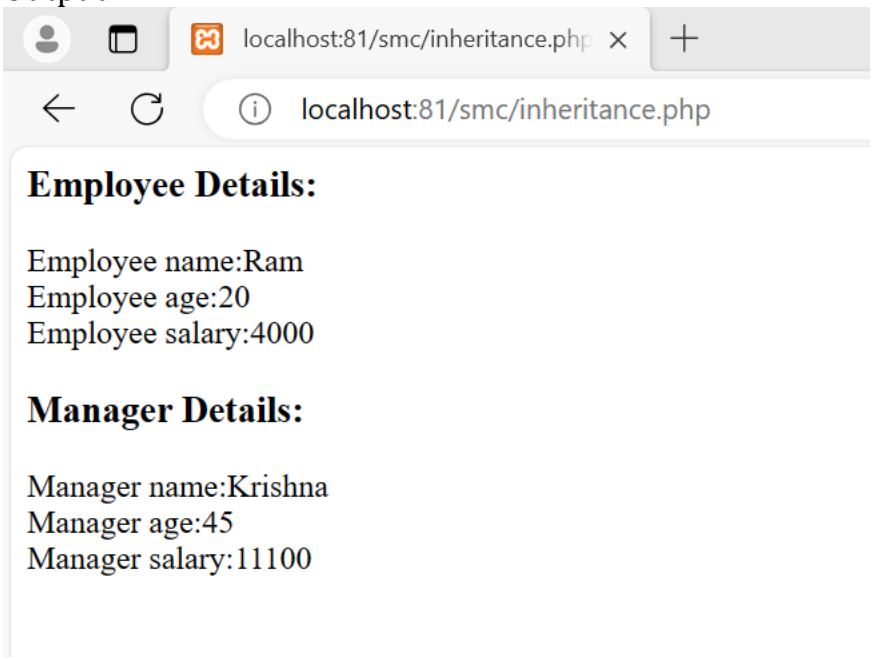
One of the main advantages of object-oriented programming is the ability to reduce code duplication with inheritance. Code duplication occurs when a programmer writes the same code more than once, a problem that inheritance strives to solve.

Example program to demonstrate inheritance in PHP.

```
<?php
class employee{
   public $name;
   public $age;
   public $salary;
   function __construct($n,$a,$s)
   {
      $this->name=$n;
      $this->age=$a;
      $this->salary=$s;
   }
   function info(){

      echo"<h3>Employee Details:</h3>";
      echo "Employee name:".$this->name."<br>";
      echo "Employee age:".$this->age."<br>";
      echo "Employee salary:".$this->salary;
   }
}
class manager extends employee{
   public $t=500;
   public $phone=600;
   public $total_salary;
   function info(){
   $this->total_salary=$this->salary+$this->t+$this->phone;

      echo"<h3>Manager Details:</h3>";
      echo "Manager name:".$this->name."<br>";
      echo "Manager age:".$this->age."<br>";
```

```
        echo "Manager salary:".$this->total_salary;
    }
}
$e1=new employee("Ram",20,4000);
$e2=new manager("Krishna",45,10000);
$e1->info();
$e2->info();
?>
```

Output:



Multilevel Inheritance in Php

- ☞ PHP supports multileve inheritance.

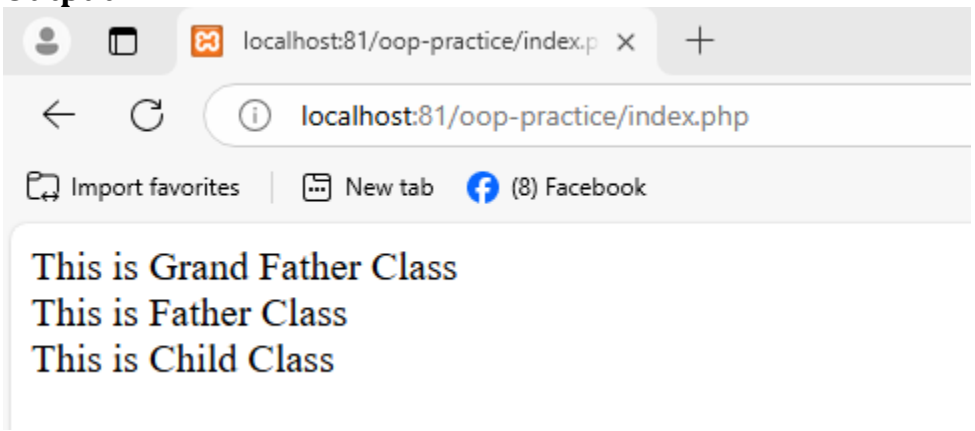
```
class class 1
{
// body
}
class class2  extends class1
{
// body
}
class class3  extends class2
{
// boddy
}
```

In the above example class2 inheriting the class1 and class3 inheriting the class2. So class3 have some properties of class2 and some properties of class1.

Example:

```
<?php
class GrandFather{
   public function showGrandFather(){
      echo"This is Grand Father Class<br>";
   }
}
class Father extends GrandFather{
   public function showFather(){
      echo"This is Father Class<br>";
   }
}
class Child extends Father{
   public function showChild(){
      echo "This is Child Class";
   }
}
$objchild=new Child();
$objchild->showGrandFather();
$objchild->showFather();
$objchild->showChild();
?>
```

Output:



Interface

- Interfaces are defined in the same way as a class, but with the interface keyword replacing the class keyword and without any of the methods having their contents defined.
- All methods declared in an interface must be public; this is the nature of an interface.
- Interfaces make it easy to use a variety of different classes in the same way. When one or more classes use the same interface, it is referred to as "polymorphism".
- Interfaces are declared with the **interface** keyword.

Characteristics of an Interface are:

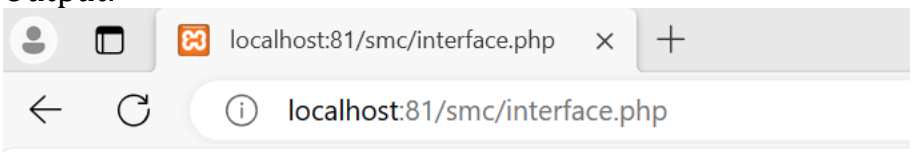
- An interface consists of methods that have no implementations, which means the interface methods are abstract methods.
- All the methods in interfaces must have public visibility scope.
- Interfaces are different from classes as the class can inherit from one class only whereas the class can implement one or more interfaces.

Example program to demonstrate interface

```
<?php
interface A{
   function add($a,$b);
}
interface B{
   function sub($c,$d);
}
interface C{
    function mul($x,$y);
 }
class D implements A,B ,C{

public function add($a,$b){
   echo "The sum:".$a+$b;
}
public function sub($c,$d){
   echo "The difference:".$c-$d;
}
public function mul($x,$y){
   echo "The Product:".$x*$y;
 }
}
```

```
$obj=new D();
$obj->add(5,3);
echo "<br>";
$obj->sub(5,3);
echo "<br>";
 $obj->mul(5,3);
?>
```

Output:



The sum:8
The difference:2
The Product:15

Access Modifier in PHP (Encapsulation/Data Hiding/Data Abstraction)

1. Example program to demonstrate **Public** access modifier

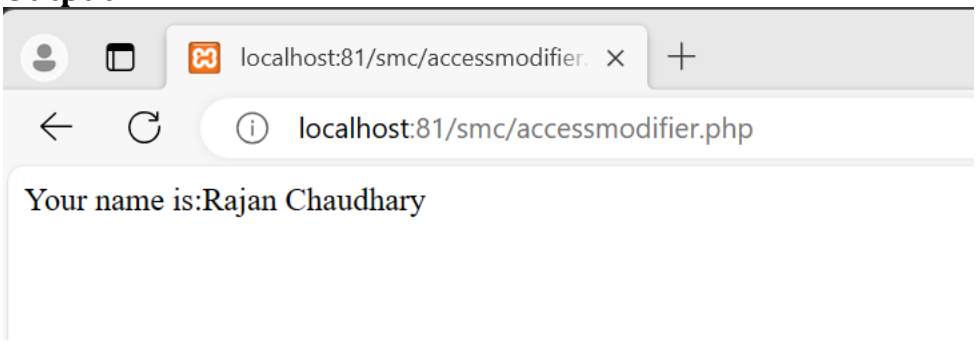
```
class A {

        public $name;
        public function show(){
        echo $this->name;
        }
}
$obj=new A();
$obj->name="Rajan Chaudhary";
$obj->show();

class B extends A{
        public function get(){

        echo "Your name is:".$this->name;
        }
```

```
}
```

Example:

```
<?php
class A{
   public $name;
   public function __construct($n)
   {
      $this->name=$n;

   }
   public function show(){
      echo "Your name is:".$this->name."<br>";
   }

   }
   $obj=new A("Rajan Chaudhary");
  // $obj->name="Kishan Safi";
   $obj->show();
?>
```

Output:



2. Example program to demonstrate **protect** access modifier.

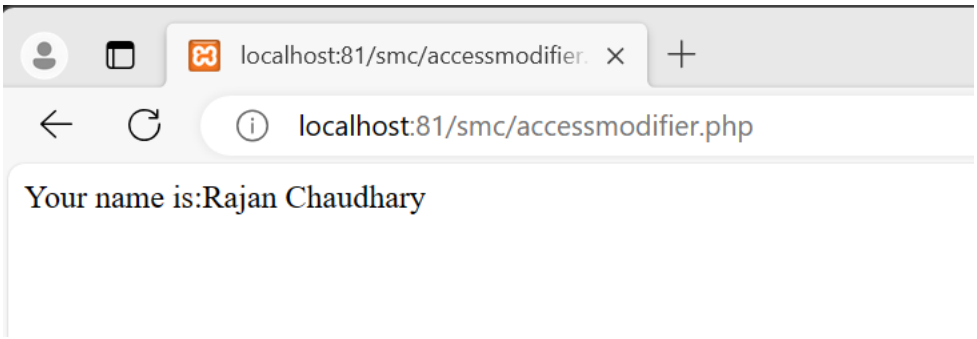
```
class A {

        protected $name;
        protected function show(){
        echo $this->name;
        }
}
$obj=new A();
$obj->name="Rajan Chaudhary";
```

```
$obj->show();

class B extends A{
        public function get(){

        echo "Your name is:".$this->name;
        }
}
```

Example:

```
<?php
class A{
   protected $name;
   public function __construct($n)
   {
      $this->name=$n;

   }
   protected function show(){
      echo "Your name is:".$this->name."<br>";
   }

   }
   class B extends A{
      public function get(){
         echo "Your name is:".$this->name."<br>";
      }

   }
   $obj=new B("Rajan Chaudhary");

   $obj->get();
?>
```

Output:



3. Example program to demonstrate private access modifier.

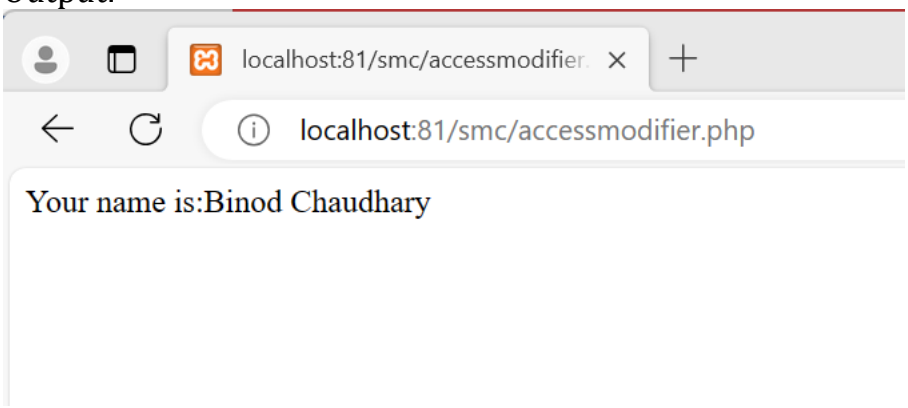
```
class A {

        private $name;
        private function show(){
        echo $this->name;
        }
}
$obj=new A();
$obj->name="Rajan Chaudhary";
$obj->show();

class B extends A{
        public function get(){

        echo "Your name is:".$this->name;
        }
}
```

Example:

```
<?php
class A{
   private $name;
   public function __construct($n)
   {
      $this->name=$n;

   }
   public function show(){
      echo "Your name is:".$this->name."<br>";
   }
```

```
  }
  class B extends A{
    public function get(){
      echo "Your name is:".$this->name."<br>";
    }

  }
  $obj=new A("Binod Chaudhary");

  $obj->show();
?>
```

Output:



	Class Itself	Outside Class	Derived Class
Public	Allow	Allow	Allow
Protected	Allow	Not Allow	Allow
Private	Allow	Not Allow	Not Allow

Abstract Class

- Abstract class is a secure class which does not have object to access property and method.
- Abstract class is declare with keyword **abstract**.
- Abstract class must have an abstract method.

Eg.

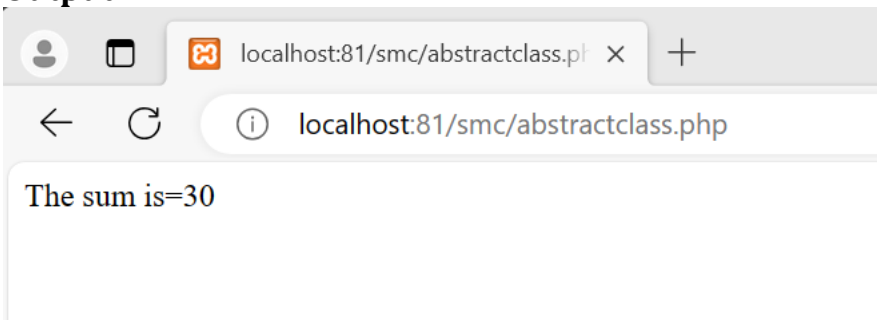
```
<?php
abstract class A{
        protected $name;
```

```
        protected function __construct($n){
                $this->name=$n;
        }
abstract protected function show();
}
class B extends A{
        public function show()
        {
                echo $this->show();
        }
}
```

Example:

```
<?php
abstract class A{
   public $name;
   abstract protected function findSum($a,$b);
}
class B extends A{
   public function findSum($x,$y){
      echo "The sum is=".$x+$y;

   }
}
$obj=new B(10,20);
$obj->findSum();
?>
```

Output:



POLYMORPHISM IN PHP

- Polymorphism derived from the Greek word Poly which means many or multiple and morphism which means forms.

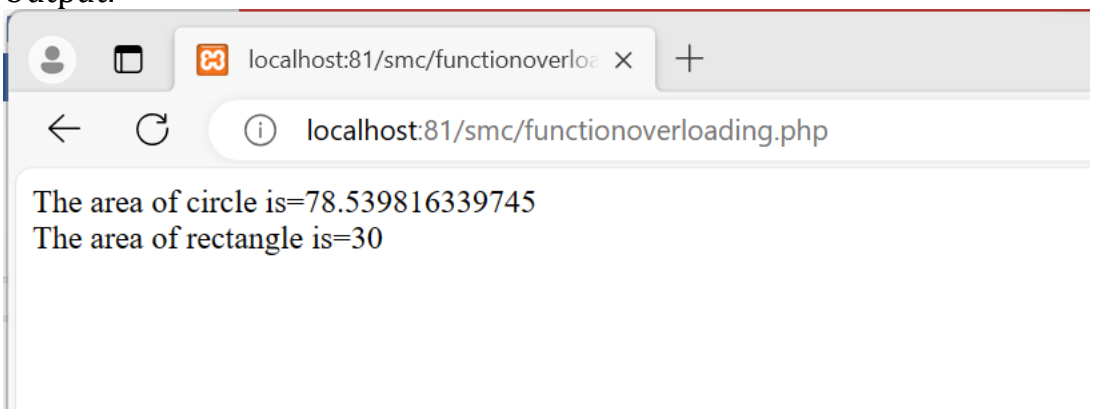
- In object oriented programming , polymorphism is **closely related to inheritance.**
- To implement polymorphism in PHP, you can use either abstract class or interfaces.
- Usually polymorphism is of two types:
  - Compile time polymorphism, which is also known as function overloading.
  - Run time polymorphism, which is also called function overriding.

Example:
Example program to demonstrate function overloading in PHP.

```
<?php
interface Shape{
   public function findArea();
}
class Circle implements Shape{
   private $radius;
public function __construct($radius)
{
   $this->radius=$radius;
}
public function findArea()
{
   return $this->radius * $this->radius * pi();
}
}
class Rectangle implements Shape{
   private $width;
   private $height;
   public function __construct($width,$height)
   {
      $this->width=$width;
      $this->height=$height;
   }
   public function findArea(){
      return $this->width*$this->height;
   }
}
$cir=new Circle(5);
$rec=new Rectangle(5,6);
```

```
echo"The area of circle is=". $cir->findArea();
echo "<br>";
echo"The area of rectangle is=". $rec->findArea();
?>
```

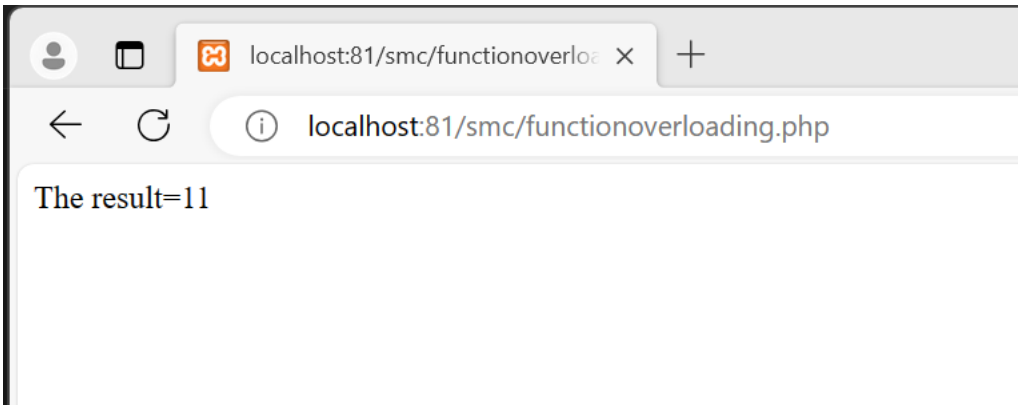
Output:



Example:
Example program to demonstrate method overriding in PHP.

```
<?php
class A{
   public $name="This is parent class";
   public function calculate($a,$b){
      return $a*$b;
   }
}
class B extends A {
   public $name="This is child class";
   public function calculate($a,$b)
   {
      return $a+$b;
   }
}
$obj=new B();
echo "The result=".$obj->calculate(5,6);
?>
```

Output:



Static members

- The **static** keyword is used to declare properties and methods of a class as static.
- Static properties and methods can be used without creating an instance of the class.
- The **static** keyword is also used to declare variables in a function which keep their value after the function has ended.

Eg.

```
Class Person{
Public static $name="Ramesh Singh";
Public static function show()
{
        echo self ::$name;
}
}
Person ::$name;
Person :: show();
```

OR

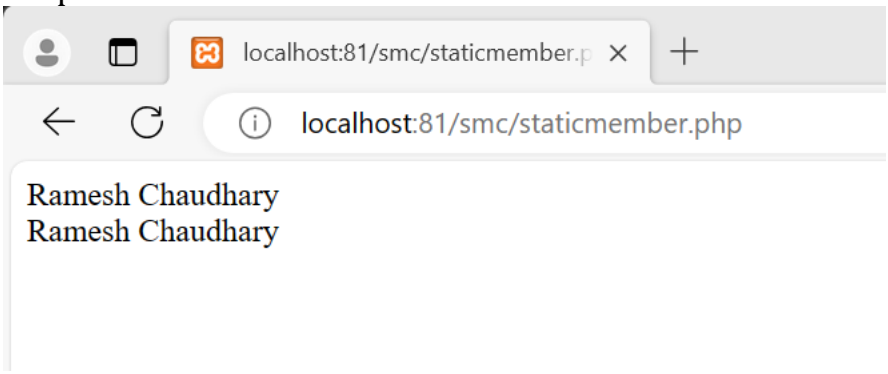
```
Class Person{
Public static $name="Ramesh Singh";
}
Class Account extends Person
{
        Public function show()
{
        echo Person ::$name;
}
```

```
}
$obj=new Account();
$obj->show();
```

Example:
Example program to demonstrate static member in php.

```
<?php
class A{
   public static $name="Ramesh Chaudhary";
   public static function show()
   {
      echo self::$name;
   }
}
echo A::$name;
echo"<br>";
A::show();
?>
```

Output:

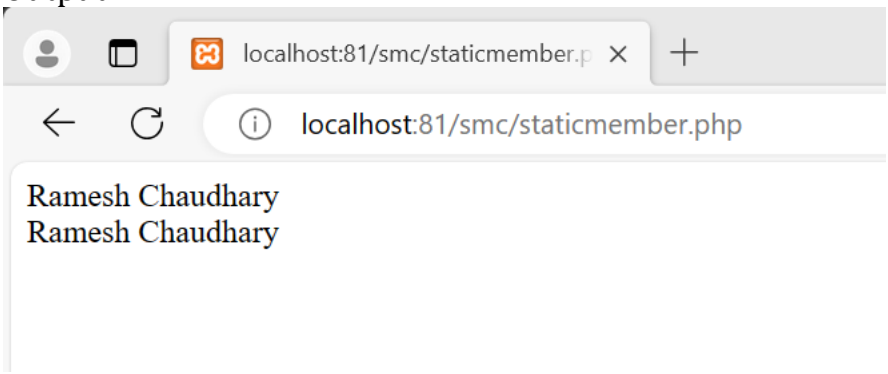


Using constructor

```
<?php
class A{
   public static $name="Ramesh Chaudhary";
   public static function show()
   {
      echo self::$name;
   }
   public function __construct()
   {
      self::show();
```

```
   }
}
$obj=new A();
echo "<br>";
$obj->show();
?>
```

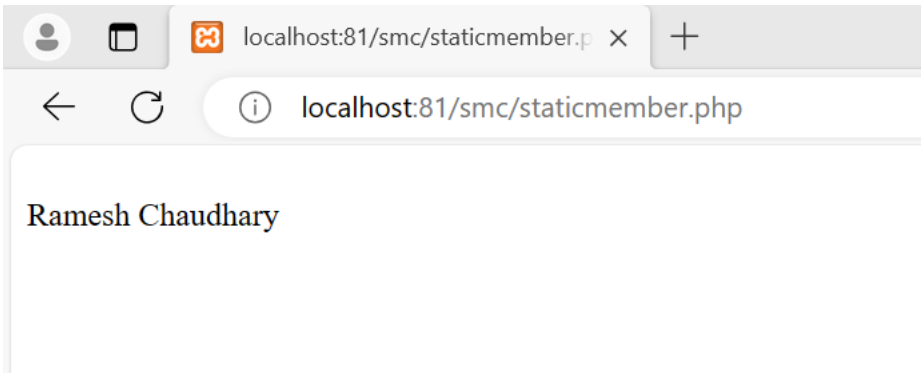
Output:



Static member in derived class

```
<?php
class A{
   public static $name="Ramesh Chaudhary";

}
class B extends A{
   public static function show()
   {
      echo A::$name;
   }
}
$obj=new B();
$obj->show();
?>
```

Output:



EXCEPTION HANDLING IN PHP

```
try
{
        if(condition)
        {
        }
        else
        {
                throw new Exception("Some message goes here");
        }
}
catch(Exception $e)
{
        echo $e->getMessage();
}
```

Note: some inbuilt up function

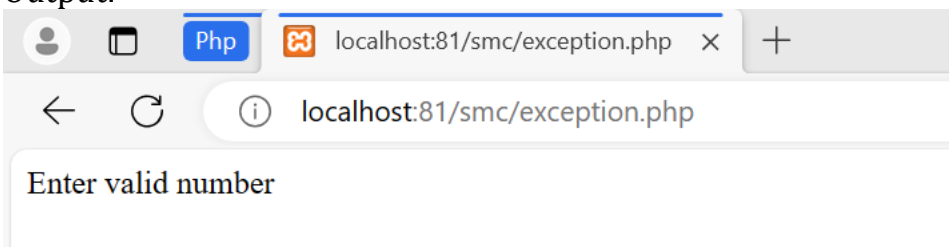
1. $e->getLine();
2. $e->getCode();
3. $e->getFile();

Example-1:

```
<?php
$n=0;
try{
  if($n<=0)
  {
    throw new Exception("Enter valid number");
  }
  $div=2/$n;
```

```
echo $div;

}catch(Exception $e)
{
   echo $e->getMessage();
}

?>
```

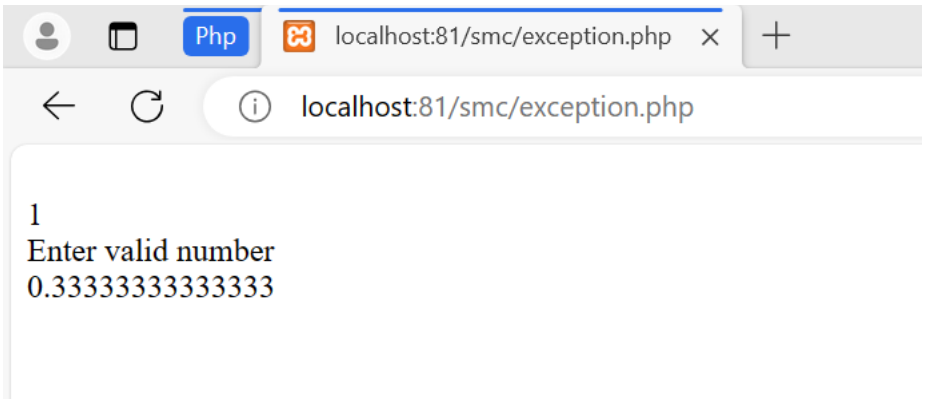
Output:



Example:2 By using function

```
<?php
function division($n)
{
   try{
      if($n<=0)
      {
         throw new Exception("<br>Enter valid number");
      }
      $div=2/$n;
   echo "<br>".$div;

   }catch(Exception $e)
   {
      echo $e->getMessage();
   }
}
division(2);
division(0);
division(6);
?>
```

Output:



User Defined Exception

```
<?php
class myException extends Exception{
   function errorMessage()
   {
      $error="My Exception message".$this->getMessage();
            "<br>Error on line no.".$this->getLine();
                  return $error;
   }
}
function division($n)
{
   try{
      if($n<=0)
      {
         throw new Exception("<br>Enter valid number");
      }
      if($n==3)
      {
         throw new myException("<br>Number is 3");
      }
      $div=2/$n;
   echo "<br>".$div;


   }catch(myException $e)
   {
      echo $e->errorMessage();
   }
```

```
    catch(Exception $e)
    {
        echo $e->getMessage();
    }
}
division(2);
division(3);
division(6);
?>
```

Output: